77-FOC 30

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。
我们开始分析过流故障。

```
void OverCurrentDetect(void)
{
    if (PWMCON1bits.FLTSTAT | PWMCON2bits.FLTSTAT | PWMCON3bits.FLTSTAT)
    {
        PWMCON1bits.FLTIEN = 0; //PWMCON1bits.CLSTAT = 0;
        PWMCON2bits.FLTIEN = 0; //PWMCON2bits.CLSTAT = 0;
        PWMCON3bits.FLTIEN = 0;

        IFS5bits.PWM1IF = 0;
        IFS5bits.PWM2IF = 0;
        IFS6bits.PWM3IF = 0;

        tCore.IoverCount ++;
        if (tCore.IoverCount > 3)
            tProtect.Code = 3;
            Error_Flag.Bits.OverCurrent = ON;
    }
    else
    {
        tCore.IoverCount = 0;
    }
}
```

对应的过流监测函数是这样的，也就是，在 PWM 中断中去查询故障状态位、因为我们是 3 个 PWM 发生器，所以呢，就查询 3 个故障状态位。如果说故障状态位是 1 的话，有一个计数值会累加，如果连续出现 3 次故障，则会置位过流故障标志位。

```
void OverCurrentDetect(void)
{
    if (PWMCON1bits.FLTSTAT | PWMCON2bits.FLTSTAT | PWMCON3bits.FLTSTAT)
    {
        PWMCON1bits.FLTIEN = 0; //PWMCON1bits.CLSTAT = 0;
        PWMCON2bits.FLTIEN = 0; //PWMCON2bits.CLSTAT = 0;
        PWMCON3bits.FLTIEN = 0;

        IFS5bits.PWM1IF = 0;
        IFS5bits.PWM2IF = 0;
        IFS6bits.PWM3IF = 0;

        tCore.IoverCount ++;
        if (tCore.IoverCount > 3)
            tProtect.Code = 3;
            Error_Flag.Bits.OverCurrent = ON;
    }
    else
    {
        tCore.IoverCount = 0;
    }
}
```

红框中的这些呢，是清除故障标志的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=====================================================================*/
void OverCurrentDetect(void)
{
    if (PWMCON1bits.FLTSTAT | PWMCON2bits.FLTSTAT | PWMCON3bits.FLTSTAT)
    {
        PWMCON1bits.FLTIEN = 0; //PWMCON1bits.CLSTAT = 0;
        PWMCON2bits.FLTIEN = 0; //PWMCON2bits.CLSTAT = 0;
        PWMCON3bits.FLTIEN = 0;

        IFS5bits.PWM1IF = 0;
        IFS5bits.PWM2IF = 0;
        IFS6bits.PWM3IF = 0;

        tCore.IoverCount ++;
        if (tCore.IoverCount > 3)
            Error_Flag.Bits.OverCurrent = ON;
    }
    else
    {
        tCore.IoverCount = 0;
    }
}
```

这里，也就是说，当连续计数次数大于 3，则程序定义的过流标志会变为 1。

```
        Motor_Flag.Bits.ReqOff = 1;

    if(Error_Flag.Bits.OverCurrent)
        tProtect.Code = 1;

    else if (Error_Flag.Bits.OverVol)
        tProtect.Code = 2;

    else if (Error_Flag.Bits.LackVol)
        tProtect.Code = 3;

    else
        tProtect.Code = 0;
}
```

变为 1 的话，就会满足这个条件，满足这个条件的话，就会停机，这样的话，就实现了过流保护。

```
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    Observer();
    CurrentLoop();
    OverCurrentDetect();
    StatusControl();
    CalSinCos(tIm.qParkAngle);
    AdcFilter();
    SpiTransmit();
    DownCounter();

}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个函数呢，会放到这里来调用，这样的话，整个故障保护的功能代码就分析完了。

这里大家需要注意的是:

1. Fault 故障非常重要!一般情况下需要提前调试好这个功能，对于避免炸机有帮助。
2. Fault 功能需要滤波，一般硬件滤波和软件滤波都需要，否则容易误触发。
3. 滤波深度 一般在 1~5us
4. Fault 触发一般需要硬件强制关闭 PWM 输出，Fault 故障解除后是否重新输出 PWM 取决于实际应用。

问：100*1nf=0.1us 0.1*5 也只有 0.5us,没有 1us 啊?

答：是的，滤波深度主要和输出功率及板子的干扰有关。课程演示板功率较小，硬件滤波比较浅，实际产品如果功率较大，这个硬件滤波时间可以 1~3us。

问：不知道软件滤波在哪里?

答：电机控制的过流都是需要硬件关断的，特别是大功率场景，软件根本来不及关断，需要靠硬件关断的，这个 Fault 引脚在单片机内部是有硬件电路直接关断 PWM 的。

下面，我们以速度环为例，来给大家讲讲参数调整的思路。

```
static void MotorSpeedLoop(void)
{
    if (Motor_Flag.Bits.SpeedLoopEnable & Motor_Flag.Bits.SpeedLoopStart)
    {
        tPid.Kp = 16384;
        tPid.Ki = tPid.Kp >> 14;
        tPid.Kd = tPid.Kp >> 12;
        tPid.sErr = (signed int)((tMotor.AimSpeed << 3) - (tMotor.ActSpeed << 3));
        tPid.sLimit = 32767;
```

也就是，要调整对应的 Kp Ki Kd。咱们这里呢，主要是用到试凑法。调试的方法呢，就是对照着目标值波形跟实际值波形来调整。那么，要看波形的话，可以借助于板载的 DAC 芯片。也就是说，我们可以使用 SPI 把 目标速度跟实际速度传送到 DAC 芯片，然后使用示波器看波形调整。

```
static void MotorSpeedLoop(void)
{
    if (Motor_Flag.Bits.SpeedLoopEnable & Motor_Flag.Bits.SpeedLoopStart)
    {
        tPid.Kp = 16384;
        tPid.Ki = tPid.Kp >> 14;
        tPid.Kd = tPid.Kp >> 12;
        tPid.sErr = (signed int)((tMotor.AimSpeed << 3) - (tMotor.ActSpeed << 3));
        tPid.sLimit = 32767;
        PidControl(&tPid);
```

也就是，把这两个变量传送出去。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void SpiDataUpdate(void)
{
    unsigned int Temp;

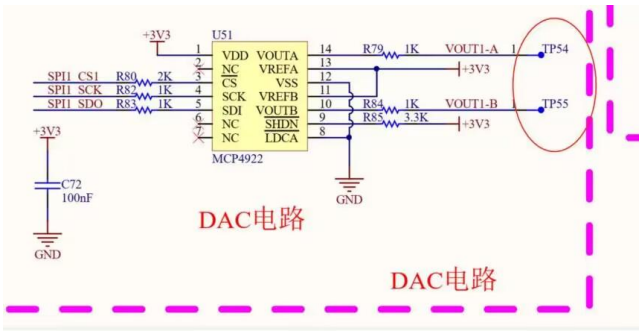
//    Temp = (unsigned int)(tObs.qIalphaEstimate + 0x8000);
//    tSpi.ReadyData[0] = Temp >> 4;
//
//    Temp = (unsigned int)(tIm.qIalpha + 0x8000);
//    tSpi.ReadyData[1] = Temp >> 4;

    tSpi.ReadyData[0] =   tMotor.AimSpeed;

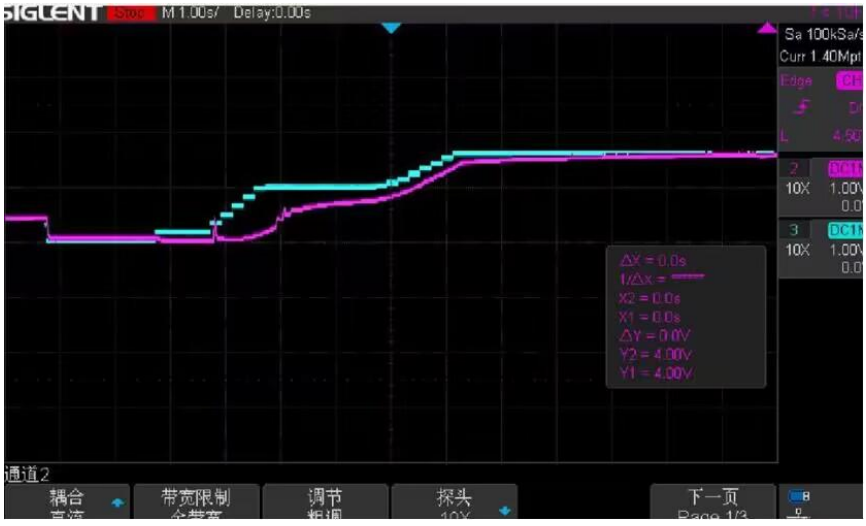
    tSpi.ReadyData[1] =  tMotor.ActSpeed;

}
```

大家看下，因为 DAC 芯片是 12 位的，我们这里电机转速最高是 2000，所以呢，这里的赋值，就可以直接写，12 位的话，数值最大是 4095。

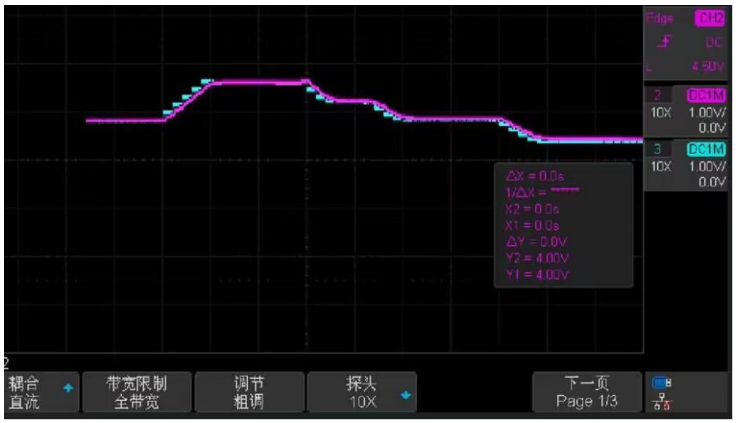


然后呢，使用示波器来看这两个通道的值。

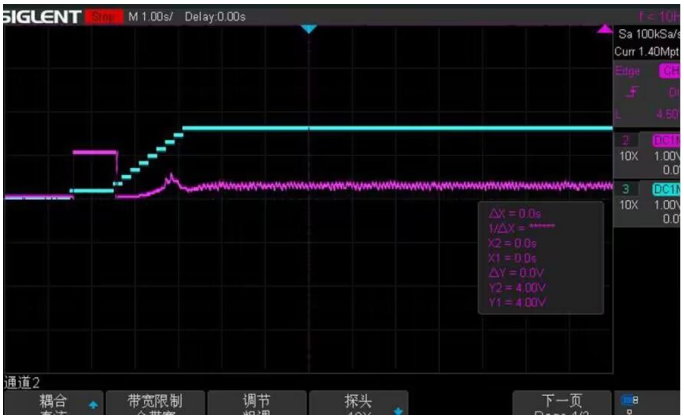


大家来看下这幅波形， 蓝色是期望转速， 红色的是实际转速。这是启动到稳定的波形因为启动阶段，要经过定位、强拖，切闭环，所以我们调整目标值的话，实际值要速度使能后才能跟踪目标值。

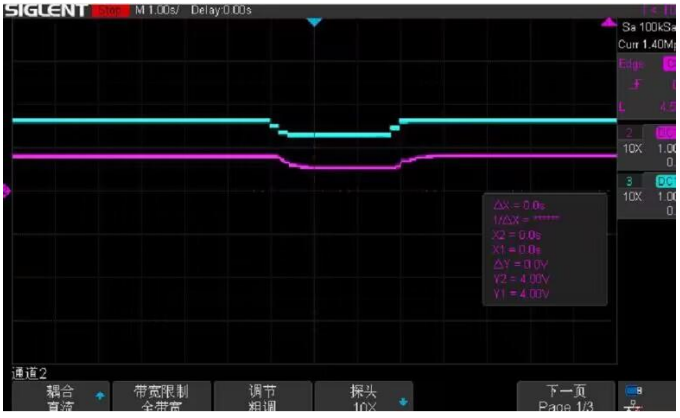
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



然后，这是进入速度闭环之后，拧动电位器，让期望值变化，实际值跟随目标值变化的情况。上图可以看出，实际值跟随目标值还是比较好的。也就是说，我们调试的话，就是调整实际值波形跟随期望值的情况，好的参数，实际值跟踪目标值很好。我们这里的试凑法调参的话，就是一个不断逼近的过程，参数放大缩小可以用二分法，最终也是可以比较容易地找到合适的参数。PID 参数呢，可以是由小变大，给了一个参数后，然后看实际效果，效果不好的话，继续调整。



我们刚开始调试呢，可以先把 Ki Kd 都设置为 0，然后慢慢加大 Kp。上面这幅图呢，是 Kp 设置 2048 Ki = 0 Kd = 0 的时候。此时呢，电机就没有转动起来，电机在抖动，说明呢，这个参数太小了。实际的速度波形呢，实际转速幅值也很低。此时呢，我们可以把 Kp 加大一些，例如设置为 8192。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

此时看到，红色波形跟踪蓝色好一些了。但是，没有完全跟上，但是，红色已经是稳定了，还没有跟上蓝色，始终有一个稳态的误差。说明参数还是有点小，我们可以继续加大一些 Kp。例如，把 Kp 设置为　16384。



此时呢，波形是这样的，红色的其实还是没有跟踪上蓝色，但是，稳态误差变小了一些。那么，根据实际调试情况来看，既然加大 Kp 有效果，则我们还是会加大 Kp。例如，变为 32767 看看。



继续加大，还是有效果，此时呢，如果电机转动还是正常的话，我们还是可以继续加大 Kp。比如，变为 60000。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



此时呢，大家可以看到，误差又缩小了，但是呢，还是有误差。但是 Kp 已经设置很大了，还是存在稳态误差的话，我们可以尝试加一点 Ki。例如，我们可以给 Ki = 8， 也就是让 Kp = 60000 Ki = 8，此时再看波形。



这里，就给了一个很小的 Ki，红色就能跟踪上蓝色了。所以，这个符合我们之前分析的积分项的作用，就是消除稳态误差，效果是明显的。

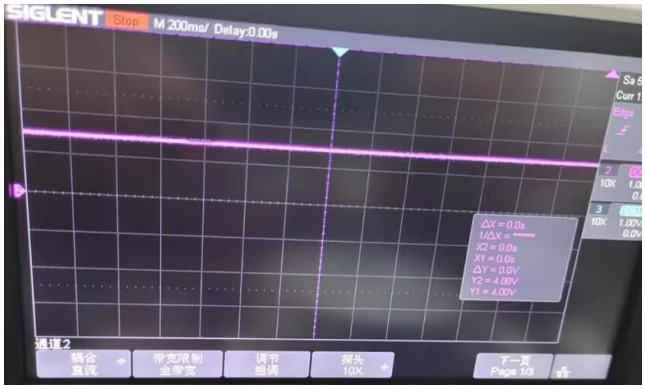


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

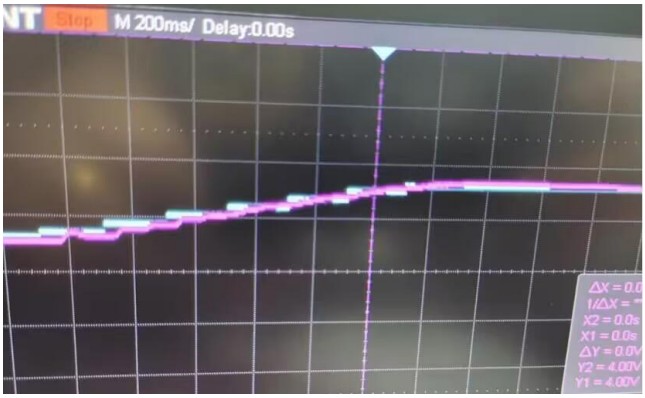
但是，这个时候我们要注意啊，虽然是加了 Ki 是跟踪上了，但是仔细看波形的话，红色波形还是有一些震荡。只不过呢，震荡幅度不算大。这个时候呢，我们可以再微调一下 Kp，把 Kp 减小一些看看。例如，把 Kp 减小到 32767。



虽然，还是可以跟踪的，但是加速过程中，还是有点震荡。



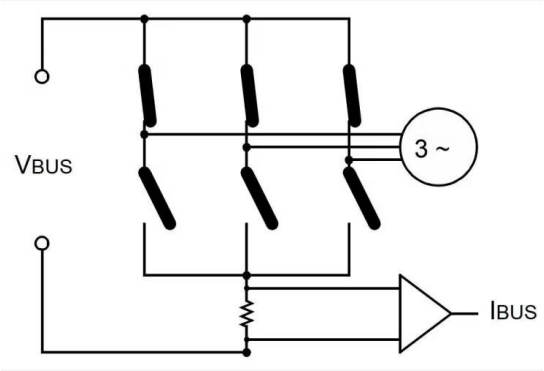
但是呢，稳态情况下是可以的。说明离我们的期望越来越接近了。这个时候呢，还是可以把 Kp 减小一些看看，例如，再变为 16384。



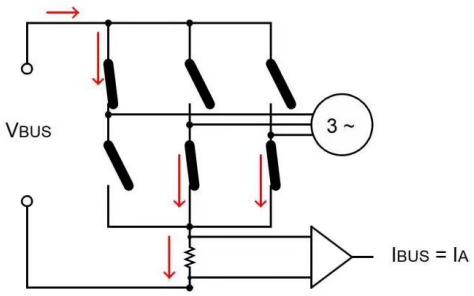
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

可以看到，震荡明显变小了，这个呢，基本可以接受了。基本调试的思路就是这样，就不断的调整，然后看波形，不管是空载，还是负载，都可以这样去调试。如果给定转速是不停动态变化的，或者有动态负载变化的话，可能要把 Kd 考虑上。给定不变，负载恒定的情况，可以不用 Kd。

上面的调试思路，大家可以动手试试看。上面关于本课程 microchip 部分的内容，我们就全部讲完了。剩下时间，再扩展和大家讲一讲单电阻采样思路。

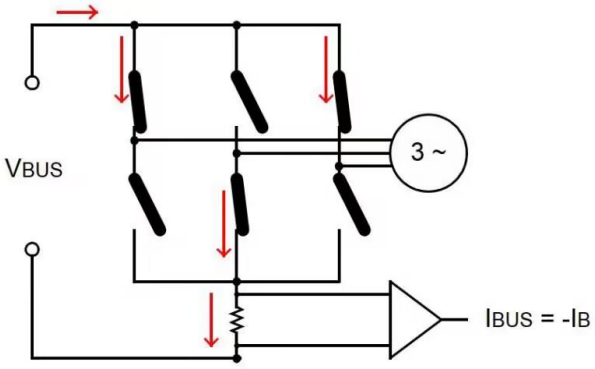


其实，单电阻的话，就是一个 PWM 周期需要两次电流采样，这样的话，才能重构出三相相电流。



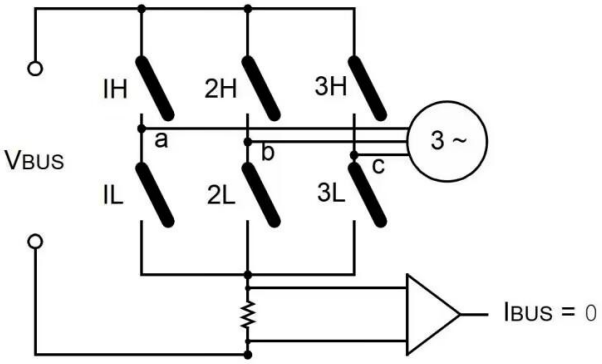
例如，这种情况下，大家看下，此时呢，上图这个矢量（1　0　0）作用后，检流电阻上的电流就对应 IA。

我们定义的是，电流流进某相绕组，则对应的相电流是正。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

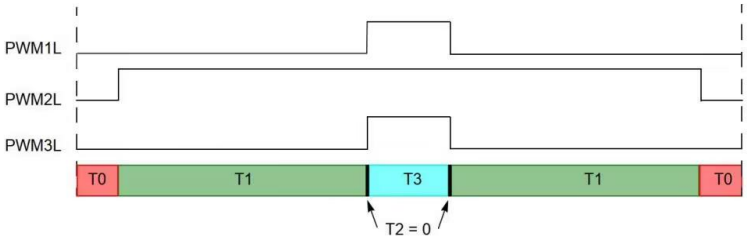
例如，这种情况，检流电阻上反映的是-IB。因为，电流是流出 B(V)相绕组。所以啊，把 6 种矢量对应的情况总结一下，就可以得出，哪个矢量作用，检流电阻上面反映的相电流是哪一个。



这里，按照这幅图做一个约定。

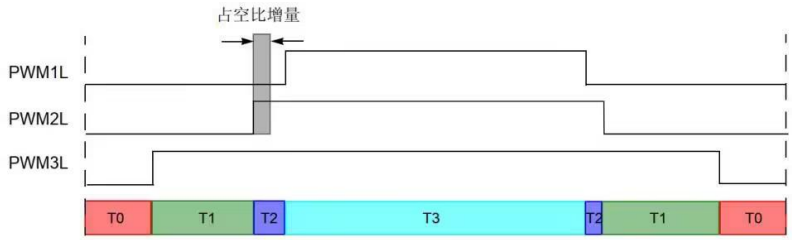
IH	2H	3H	1L	2L	3L	IBUS
ON	OFF	OFF	OFF	ON	ON	+IA
OFF	ON	OFF	ON	OFF	ON	+IB
OFF	OFF	ON	ON	ON	OFF	+IC
OFF	ON	ON	ON	OFF	OFF	–IA
ON	OFF	ON	OFF	ON	OFF	–IB
ON	ON	OFF	OFF	OFF	ON	–IC

我们就可以得出这张表，这样的话，基础重构三相电流的思路，也就清楚了。所以，对于单电阻采样来说，采样点的话，需要去变化，在一个 PWM 周期内，需要采样两次。但是呢，这里有需要注意的点，因为矢量的作用时间是变化的，矢量在旋转，这样的话，就会碰到 PWM 波形有相似占空比的情况。



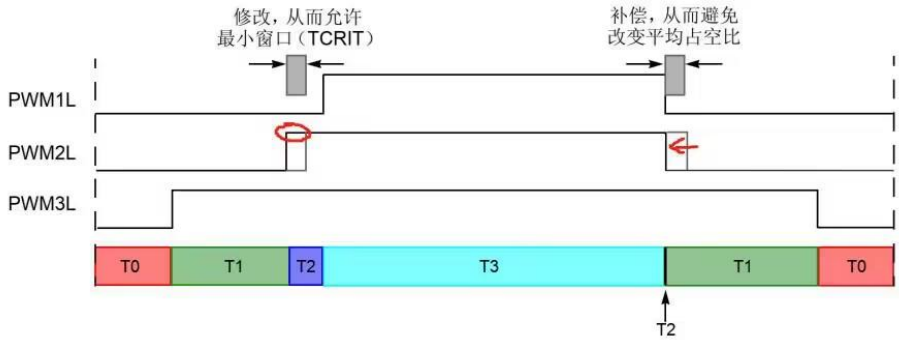
例如，矢量 1 作用时间很大，矢量 2 作用的时间很小。这样的话，有一相相电流的采样窗口很小，就会出问题。此时呢，一般的策略就是，要补偿占空比。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

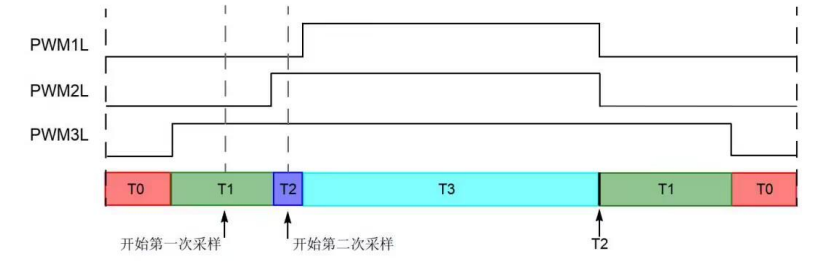


例如，这样去补偿，补偿一个能够采样到相电流的最小占空比增量。一般采取的策略是，PWM 的前半周补偿，然后后半周要减掉之前的补偿量。这样的话，才能保持整体的 PWM 波跟之前的等效。

但是呢，正是因为前半周补偿了，可以采样到正确的相电流，后半周减去补偿，也不影响电流采样，所以呢，这个办法也是可取的。



大家看，对于 PWM2L 的话，左半部分修改占空比大小，然后右边补偿回来。也就是左边增加了占空比，右边减小占空比。这样保持整个周期的占空比是不变的，只是对 PWM 波形做了变形处理。



这个波形呢，就是补偿之后的采样情况，能够反映一个 PWM 周期有两次采样。那么这两次采样不是同时的，大家想一想，这时候 IA + IB + IC = 0 还成立么？

电机电感量越大，电流变化越小。载频越高，电流变化越小。如果电机的电感量比较小，使用单电阻方案，这里可能就会有点问题。对于电感量较大的高压电机，是比较适用单电阻采样。单电阻方案节约成本，增加了控制代码，对 MCU 的运算速度和采样时间稍有要求
对单电阻控制感兴趣的学员可以自己去尝试一下。

关于 FOC 课程的内容我们已经讲完了，下面大约花 2 次课，我们介绍一下张飞单片机部分的内容，感兴趣的学员可关注一下，本课程有什么问题也可以在下面 2 次晚课上问我。

谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。